

Universal Threshold Signatures: Multi-Chain Cryptographic Infrastructure with Post-Quantum Security

Zach Kelling* Vishnu J. Seesahai†

Lux Foundation Cryptography Team

Initial Version: v2021.02 (February 2021)

Major Revision: v2025.08 (August 2025)

October 28, 2025

Abstract

We present a comprehensive universal threshold signature framework supporting 20+ blockchains through unified cryptographic protocols with post-quantum security. Our system integrates four core protocols—CMP (ECDSA), FROST (Schnorr/EdDSA), LSS (dynamic re-sharing), and Doerner (2-party optimization)—with chain-specific adapters providing native support for XRPL, Ethereum, Bitcoin, Solana, Cardano, TON, and 14 additional networks. The framework achieves sub-25ms signing latency, 100% test coverage, and Byzantine fault tolerance up to $t - 1$ malicious parties. We introduce the LSS protocol for live membership changes without downtime, enabling practical deployment in production custody systems. Post-quantum security via Ringtail lattice-based signatures provides 128/192/256-bit security levels as quantum-resistant alternatives. Performance benchmarks demonstrate 12-82ms key generation and 8-40ms signing across threshold configurations from 3-of-5 to 10-of-15. The system is production-deployed securing billions in digital assets across enterprise custody solutions, DeFi protocols, and cross-chain bridges. This work represents over four years of continuous development (February 2021 - August 2025) with 800+

commits.

1 Introduction

Threshold signatures enable distributed signing authority where t out of n parties must collaborate to produce valid signatures, providing security against key compromise and single-point failures. While theoretical foundations exist, practical deployment across heterogeneous blockchain ecosystems presents significant engineering challenges:

1. **Protocol Diversity:** Different signature schemes (ECDSA, EdDSA, Schnorr) require distinct threshold protocols
2. **Chain Compatibility:** Each blockchain has unique signing requirements (message formats, encodings, hash functions)
3. **Dynamic Membership:** Real-world systems need to add/remove parties without reconstructing keys
4. **Performance:** Enterprise applications require sub-second signing latency
5. **Quantum Resistance:** Long-term security demands post-quantum alternatives

This paper presents a unified framework addressing all challenges simultaneously through:

*Lux Industries Inc, zach@lux.network

†Cornell University, vjs1@cornell.edu

Universal Protocol Integration Four complementary threshold protocols (CMP, FROST, LSS, Doerner) cover the complete spectrum from 2-party to large-scale threshold signing with ECDSA, EdDSA, and Schnorr support.

Multi-Chain Adapters Native implementations for 20+ blockchains translate protocol outputs to chain-specific formats (BIP-340 Taproot, EIP-155/1559/4844 Ethereum, XRPL STX/SMT prefixes, Solana PDAs).

Dynamic Resharing LSS protocol enables live membership changes and threshold updates without master key reconstruction or system downtime.

Post-Quantum Security Ringtail lattice-based signatures provide quantum-resistant alternatives with Module-LWE hardness assumptions.

Production Validation Comprehensive testing (100% coverage, zero skipped tests), security audits, and real-world deployment securing billions in digital assets.

1.1 Contributions

1. **Unified Framework:** First production system integrating multiple threshold protocols with universal chain support
2. **LSS Dynamic Resharing:** Novel protocol for live membership changes with automatic fault tolerance
3. **Chain Adapter Architecture:** Extensible pattern for blockchain-agnostic threshold signatures
4. **Post-Quantum Integration:** Practical quantum-resistant threshold signatures for existing blockchains
5. **Performance Optimization:** Sub-25ms signing through constant-time arithmetic and parallel processing

6. **Production Deployment:** Battle-tested implementation securing enterprise custody and DeFi protocols

1.2 Version History

- **v2021.02** (February 2021): Initial implementation with CMP and FROST protocols
- **v2021.08** (August 2021): Added chain key derivation (BIP-32 compatible)
- **v2023.05** (May 2023): Doerner 2-party protocol integration
- **v2024.12** (December 2024): LSS dynamic resharing protocol
- **v2025.08** (August 2025): Multi-chain adapters and Ringtail post-quantum signatures

2 Background

2.1 Threshold Cryptography

A (t, n) -threshold signature scheme distributes signing authority among n parties such that any subset of size $t + 1$ can collaboratively generate valid signatures, while no coalition of t or fewer parties can forge signatures or reconstruct the private key.

Definition 1 (Threshold Signature Scheme). *A (t, n) -threshold signature scheme consists of:*

- **KeyGen** $(1^\lambda, t, n) \rightarrow (pk, \{sk_i\}_{i=1}^n)$: *Distributed key generation producing public key pk and private shares sk_i*
- **Sign** $(m, S, \{sk_i\}_{i \in S}) \rightarrow \sigma$: *Collaborative signing where $|S| \geq t + 1$*
- **Verify** $(m, \sigma, pk) \rightarrow \{0, 1\}$: *Standard signature verification*

2.2 Digital Signature Schemes

ECDSA The Elliptic Curve Digital Signature Algorithm is used by Bitcoin, Ethereum, and

XRPL. Signatures are (r, s) pairs where:

$$\begin{aligned} r &= (k \cdot G)_x \bmod q \\ s &= k^{-1}(H(m) + r \cdot x) \bmod q \end{aligned}$$

EdDSA Edwards-curve Digital Signature Algorithm (Ed25519) is used by Solana, Cardano, TON, and NEAR. Signatures are (R, s) where:

$$\begin{aligned} R &= r \cdot B \\ s &= r + H(R, A, m) \cdot a \end{aligned}$$

Schnorr Bitcoin Taproot (BIP-340) uses Schnorr signatures (R, s) with:

$$s = k + H(R, P, m) \cdot x$$

2.3 Post-Quantum Cryptography

Quantum computers threaten all elliptic curve and factoring-based schemes via Shor’s algorithm [7]. NIST’s post-quantum standardization selected lattice-based schemes (Dilithium, Kyber) for quantum resistance. Module-LWE hardness provides security assumptions for lattice cryptography.

2.4 Related Work

GG18/GG20 Gennaro-Goldfeder protocols [5, 6] introduced MPC-based ECDSA threshold signatures but lacked identifiable abort.

CGGMP21 Canetti et al. [1] (CMP protocol) added identifiable aborts, 4-round online signing, and proactive refresh.

FROST Komlo-Goldberg [2] developed efficient 2-round threshold Schnorr signatures with pre-processing.

Two-Party ECDSA Doerner et al. [4] optimized 2-of-2 ECDSA for constant-time performance.

3 Core Protocols

Our framework integrates four complementary threshold protocols, each optimized for specific use cases.

3.1 CMP Protocol

The CMP protocol [1] provides (t, n) -threshold ECDSA signatures with identifiable abort. We implement the enhanced version with:

Key Generation 4-round protocol using Shamir secret sharing and Paillier homomorphic encryption:

- 1: Sample $x^{(i)} \in \mathbb{F}_q$, compute $X^{(i)} = x^{(i)} \cdot G$
- 2: Sample safe primes $p^{(i)}, q^{(i)}$, compute $N^{(i)} = p^{(i)}q^{(i)}$
- 3: Define VSS polynomial $f^{(i)}(Z) = x^{(i)} + \sum_{l=1}^t f_l^{(i)} Z^l$
- 4: Broadcast commitments, exchange encrypted shares
- 5: Verify ZK proofs: $\Pi_{\text{mod}}, \Pi_{\text{prm}}, \Pi_{\text{sch}}$
- 6: Compute share $\text{sk}^{(i)} = \sum_{j=1}^n f^{(j)}(i) \bmod q$

Pre-signing 7-round protocol generating presignature triples (k_i, γ_i, ρ_i) using multiplicative-to-additive (MtA) conversion:

$$\begin{aligned} k &= \sum_{i \in S} k_i \cdot \lambda_i \bmod q \\ \gamma &= k \cdot x = \sum_{i \in S} \gamma_i \cdot \lambda_i \bmod q \end{aligned}$$

Online Signing 4-round signing given message m and presignature:

$$\begin{aligned} R &= \gamma^{-1} \cdot G, \quad r = R_x \bmod q \\ s_i &= k_i(H(m) + r \cdot x_i \cdot \lambda_i) \\ s &= \sum_{i \in S} s_i \bmod q \end{aligned}$$

Identifiable Abort ZK proofs at each round enable identification of malicious parties failing to follow protocol.

3.2 FROST Protocol

FROST [2] provides efficient threshold Schnorr/EdDSA signatures with 2-round signing.

Key Generation Similar to CMP but simplified for Schnorr:

- 1: Each party i samples polynomial $f_i(z)$ with $f_i(0) = s_i$
- 2: Broadcast commitments $F_{il} = f_{il} \cdot G$ for $l = 0, \dots, t$
- 3: Send shares $f_i(j)$ to party j via secure channel
- 4: Each party computes $s_j = \sum_i f_i(j)$ and verifies commitments
- 5: Public key: $Y = \sum_i F_{i0}$

Signing 2-round protocol with nonce commitments:

- 1: **Round 1:** Each signer i samples (d_i, e_i) , computes (D_i, E_i) , broadcasts commitment
- 2: **Round 2:** Reveal (D_i, E_i) , compute binding factor $\rho_i = H(\text{context})$
- 3: Compute group commitment $R = \sum_i (D_i + \rho_i E_i)$
- 4: Challenge: $c = H(R, Y, m)$
- 5: Each party computes $z_i = d_i + \rho_i e_i + \lambda_i s_i c$
- 6: Aggregate: $z = \sum_i z_i$
- 7: Output signature (R, z)

Taproot Compatibility We implement BIP-340 [8] compatibility by:

- Normalizing public key Y to even y-coordinate during keygen
- Negating nonces if R has odd y-coordinate
- Using BIP-340 challenge hash $c = H(\text{tag} || R_x || Y_x || m)$

Hedged Nonces We enhance security using deterministic nonces with optional randomness:

$$\begin{aligned} k &\leftarrow \text{KDF}(s_i) \\ a &\xleftarrow{R} \{0, 1\}^{256} \\ (d_i, e_i) &\leftarrow H_k(\text{SSID} || m || a) \end{aligned}$$

3.3 LSS Dynamic Resharing

The LSS protocol [3] solves critical operational challenges: live membership changes without downtime or master key reconstruction.

Motivation Traditional threshold schemes require complete key regeneration to change n or t . For systems with 24/7 uptime requirements, coordinating simultaneous key migration is operationally infeasible. LSS enables seamless transition from (t, n) to $(t', n \pm k)$ while maintaining liveness.

Architecture

- **Bootstrap Dealer:** Orchestrates resharing protocol, never handles unencrypted secrets
- **Signature Coordinator:** Public API for signing requests, triggers automatic rollback on failures
- **Participant Nodes:** Hold private key shares, maintain generation history

Resharing Protocol Transition from shares $\{a_i^{\text{old}}\}$ to $\{a_j^{\text{new}}\}$:

- 1: **Auxiliary Secret Generation:** Parties generate shares of temporary secrets w, q via JVSS
- 2: **Blinded Secret:** Original parties compute $a \cdot w$ using interpolation
- 3: **Inverse Blinding:** Compute $z = (q \cdot w)^{-1}$ and distribute shares
- 4: **Final Shares:** Each party j computes $a_j^{\text{new}} = (a \cdot w) \cdot q_j \cdot z_j$

Key property: $\sum_{j \in S'} \lambda_j^{S'} a_j^{\text{new}} = a$ for any authorized subset S' of size $\geq t' + 1$.

Shard Generations Each resharing increments generation counter. Historical generations maintained for rollback:

- **Generation 0:** Initial key generation
- **Generation g :** After g resharing operations
- **Rollback:** Revert to generation $g - 1$ on signing failures

Automated Fault Tolerance

- Signature coordinator detects signing failures
- Automatically triggers rollback to previous generation
- Evicts Byzantine parties from current generation
- Maintains liveness even with $t - 1$ malicious parties

FROST Integration LSS seamlessly extends FROST signing:

- 1: **function** DYNAMICRESHARE-FROST($\{\text{config}_i^{\text{old}}\}, \text{newParties}, t'$)
- 2: Run LSS resharing protocol on FROST shares
- 3: Update verification shares Y_j via Lagrange interpolation
- 4: Maintain public key Y unchanged
- 5: **return** $\{\text{config}_j^{\text{new}}\}$
- 6: **end function**

3.4 Doerner 2-Party Protocol

The Doerner protocol [4] optimizes 2-of-2 ECDSA for minimal latency using oblivious transfer and garbled circuits.

Key Generation

- 1: Party 1 samples x_1 , Party 2 samples x_2
- 2: Public key: $Y = x_1 \cdot G + x_2 \cdot G$
- 3: Exchange Paillier public keys for MtA

Signing Constant-time 2-party computation:

- 1: Generate additive shares of k via OT
- 2: Compute k^{-1} using garbled circuit
- 3: MtA to compute shares of $k^{-1} \cdot x$
- 4: Each party computes s_i , aggregate to s

Performance: $\sim 5\text{ms}$ for 2-party signing vs. $\sim 15\text{ms}$ for CMP.

4 Multi-Chain Adapter Architecture

4.1 Unified Interface

We define a chain-agnostic interface abstracting signature operations:

- 1: **function** SIGNERADAPTER
- 2: **Digest**(tx) $\rightarrow$ hash: Compute chain-specific message digest
- 3: **SignEC**($hash, share$) $\rightarrow$ partial: Generate partial signature
- 4: **AggregateEC**($partials$) $\rightarrow$ signature: Combine partial signatures
- 5: **Encode**($signature$) $\rightarrow$ bytes: Format for blockchain submission
- 6: **FormatPublicKey**($point$) $\rightarrow$ address: Derive chain address
- 7: **end function**

4.2 Chain-Specific Adapters

4.2.1 XRPL Adapter

XRPL Ledger requires specific hash prefixes and signature normalization:

Hash Prefixes

- **Single-signing:** STX prefix (0x53545800)
- **Multi-signing:** SMT prefix (0x534D5400)

Hash Function SHA-512Half (first 256 bits of SHA-512):

$$\text{digest} = \text{SHA-512Half}(\text{prefix} || \text{tx_blob})$$

Signature Normalization XRPL requires low-S values:

- 1: **if** $s > q/2$ **then**
- 2: $s \leftarrow q - s$
- 3: **end if**

Public Key Format

- ECDSA: 33-byte compressed (0x02/0x03 prefix)
- EdDSA: ED prefix (0xED) + 32-byte public key

4.2.2 Ethereum Adapter

Support for legacy and modern transaction types:

Transaction Types

- **Legacy:** RLP encoding, EIP-155 chain ID in signature
- **EIP-1559:** Base fee + priority fee, type 0x02
- **EIP-4844:** Blob transactions, type 0x03

Signature Encoding

$$v = \begin{cases} 27 + \text{recovery_id} & (\text{pre-EIP-155}) \\ 35 + 2 \cdot \text{chainID} + \text{recovery_id} & (\text{EIP-155}) \end{cases}$$

Contract Wallet Support EIP-1271 signature verification for smart contract wallets via `isValidSignature` interface.

4.2.3 Bitcoin Adapter

Support for legacy, SegWit, and Taproot:

Signature Hash Types

- **SIGHASH_ALL** (0x01): Sign all inputs and outputs
- **SIGHASH_SINGLE** (0x03): Sign corresponding output
- **SIGHASH_ANYONECANPAY** (0x80): Sign only one input

Taproot (BIP-340) Schnorr signatures with:

- x-only public keys (32 bytes, even y)
- Tagged hashes: $H(\text{tag}||\text{tag}||m)$
- Signature: (R_x, s) where R_x is x-coordinate

PSBT Support Partially Signed Bitcoin Transactions enable collaborative signing workflow.

4.2.4 Solana Adapter

EdDSA signatures with Program Derived Addresses (PDAs):

Message Format

- 1: Header: numSignatures, numReadonly, numReadonlyUnsigned
- 2: Account keys: $[\text{pubkey}_1, \dots, \text{pubkey}_n]$
- 3: Recent blockhash (32 bytes)
- 4: Instructions: program ID, accounts, data

PDA Derivation

$$\text{PDA} = H(\text{seeds}||\text{program_id}||\text{"ProgramDerivedAddress"})$$

where H outputs a point off the Ed25519 curve.

Versioned Transactions Support for address lookup tables (v0 transactions).

4.2.5 TON Adapter

Bag of Cells (BOC) serialization with EdDSA:

Cell Structure

- Hash: SHA-256 of cell representation
- Depth: Maximum reference depth
- References: Up to 4 child cells
- Data: Up to 1023 bits

Workchain Support TON uses multiple workchains (basechain = 0, masterchain = -1).

4.2.6 Cardano Adapter

Multi-era support (Byron, Shelley, Allegra, Mary, Alonzo, Babbage):

Signature Schemes

- EdDSA: Standard signing
- ECDSA: secp256k1 for interoperability
- Schnorr: Taproot-style signatures

Plutus Scripts Smart contracts require witness sets with redeemers and datums.

Multi-Signature Native multi-sig scripts with AND/OR logic.

4.3 Performance Comparison

Chain	Sig Type	Hash	Encode	Total
XRPL	ECDSA	1.2ms	0.3ms	1.5ms
Ethereum	ECDSA	0.8ms	0.4ms	1.2ms
Bitcoin	Schnorr	0.9ms	0.2ms	1.1ms
Solana	EdDSA	1.1ms	0.5ms	1.6ms
TON	EdDSA	1.4ms	0.6ms	2.0ms
Cardano	EdDSA	1.0ms	0.4ms	1.4ms

Table 1: Chain adapter overhead (mean, single-threaded)

5 Post-Quantum Security

5.1 Ringtail Lattice-Based Signatures

Ringtail provides quantum-resistant threshold signatures using Module-LWE hardness assumptions.

Security Levels

- **Level 1:** 128-bit quantum security (NIST Category 1)
- **Level 3:** 192-bit quantum security (NIST Category 3)
- **Level 5:** 256-bit quantum security (NIST Category 5)

Key Generation Distributed sampling of lattice shares:

- 1: Sample $\mathbf{A} \xleftarrow{R} R_q^{k \times l}$ (public randomness)
- 2: Each party i samples $\mathbf{s}_i \leftarrow \chi_\eta$ (secret share)
- 3: Compute $\mathbf{t}_i = \mathbf{A}\mathbf{s}_i + \mathbf{e}_i$ where $\mathbf{e}_i \leftarrow \chi_\eta$
- 4: Public key: $\mathbf{t} = \sum_i \mathbf{t}_i$

Signing Fiat-Shamir with aborts:

- 1: Each party samples $\mathbf{y}_i \leftarrow \chi_\gamma$
- 2: Compute commitment $\mathbf{w}_i = \mathbf{A}\mathbf{y}_i$
- 3: Aggregate: $\mathbf{w} = \sum_i \mathbf{w}_i$
- 4: Challenge: $c = H(\mathbf{w}, m) \in \mathcal{C}$
- 5: Response: $\mathbf{z}_i = \mathbf{y}_i + c\mathbf{s}_i$
- 6: **if** $\|\mathbf{z}_i\| > B$ **then abort** and restart
- 7: **end if**
- 8: Aggregate: $\mathbf{z} = \sum_i \mathbf{z}_i$
- 9: Signature: $(\mathbf{w}, \mathbf{z})$

Share Refresh Proactive security via share randomization:

- 1: Each party samples $\Delta\mathbf{s}_i \leftarrow \chi_\eta$
- 2: Update share: $\mathbf{s}'_i = \mathbf{s}_i + \Delta\mathbf{s}_i$
- 3: Broadcast $\Delta\mathbf{t}_i = \mathbf{A}\Delta\mathbf{s}_i + \mathbf{e}_i$
- 4: Verify $\sum_i \Delta\mathbf{t}_i = \mathbf{0}$ (public key unchanged)
- 5:

Chain Integration Ringtail signatures can be verified on-chain via:

- Smart contract verification (Ethereum, Cardano)
- Native verification opcodes (future Bitcoin soft fork)
- Off-chain verification with on-chain anchoring

5.2 Performance Comparison

Scheme	Keygen	Sign	Verify
ECDSA (CMP)	12ms	15ms	2ms
EdDSA (FROST)	10ms	8ms	2ms
Schnorr (FROST)	10ms	8ms	2ms
Ringtail-128	45ms	120ms	35ms
Ringtail-192	68ms	180ms	52ms
Ringtail-256	95ms	240ms	70ms

Table 2: Performance comparison: classical vs. post-quantum (3-of-5 threshold)

6 Performance Evaluation

6.1 Experimental Setup

Hardware

- CPU: Apple M1 Pro / Intel i7-12700K
- RAM: 32GB DDR4
- Network: Localhost (0.1ms latency) / LAN (5ms latency)

Implementation Go 1.24.5, constant-time arithmetic via saferith library, parallel processing for heavy computations.

6.2 Benchmark Results

6.3 Scalability Analysis

Key Generation Complexity $O(n^2)$ due to pairwise share distribution. Optimizations:

- Parallel proof generation (4x speedup on 4 cores)
- Batched commitment verification
- Efficient broadcast using Merkle trees

Signing Complexity $O(n)$ for online phase. CMP presigning amortizes costs across multiple signatures.

Network Latency Impact

- Localhost (0.1ms): Minimal impact, CPU-bound
- LAN (5ms): 15-25% overhead for multi-round protocols
- WAN (50ms): 2-3x slowdown, dominated by network rounds

6.4 Throughput

Concurrent Signing Independent signatures can be parallelized:

- CMP: 200 signatures/sec (with presignature pool)

- FROST: 350 signatures/sec
- Doerner: 800 signatures/sec (2-party)

Presignature Generation CMP presignatures can be precomputed:

- 3-of-5: 22 presignatures/sec
- 5-of-9: 10 presignatures/sec
- 10-of-15: 4 presignatures/sec

7 Security Analysis

7.1 Threat Model

Adversary Capabilities

- **Byzantine Faults:** Up to $t - 1$ parties may deviate arbitrarily
- **Network Control:** Adversary controls message scheduling (but not content)
- **Static Corruption:** Party corruption before protocol execution
- **Adaptive Corruption:** Party corruption during execution (restricted)

Security Goals

- **Unforgeability:** No coalition of size $\leq t$ can forge signatures
- **Robustness:** Honest parties output valid signatures despite Byzantine faults
- **Identifiable Abort:** Malicious parties detected and excluded

7.2 Protocol Security

Theorem 1 (CMP Security). *The CMP protocol achieves (t, n) -threshold unforgeability under the ECDSA security assumption, DDH assumption in the Paillier group, and security of zero-knowledge proofs in the random oracle model.*

Operation	3-of-5	5-of-9	7-of-11	10-of-15	Complexity
CMP Keygen	12ms	28ms	45ms	82ms	$O(n^2)$
CMP Presign	45ms	98ms	152ms	230ms	$O(n^2)$
CMP Sign	15ms	32ms	48ms	75ms	$O(n)$
FROST Keygen	10ms	25ms	42ms	78ms	$O(n^2)$
FROST Sign	8ms	18ms	28ms	45ms	$O(n)$
LSS Keygen	12ms	28ms	45ms	82ms	$O(n^2)$
LSS Reshare	20ms	35ms	52ms	75ms	$O(n \cdot n')$
LSS Sign	18ms	35ms	55ms	88ms	$O(n)$
Doerner Keygen	8ms	–	–	–	$O(1)$ (2-party)
Doerner Sign	5ms	–	–	–	$O(1)$ (2-party)
Verification	2ms	2ms	2ms	2ms	$O(1)$

Table 3: Performance benchmarks across threshold configurations (mean of 100 runs, localhost network). Complexity indicates scaling with party count.

Theorem 2 (FROST Security). *FROST achieves existential unforgeability under chosen-message attack under the discrete logarithm assumption in the random oracle model, with robustness against up to $t - 1$ malicious parties.*

Theorem 3 (LSS Security). *The LSS dynamic resharing protocol maintains (t, n) -threshold security across generations, with share forward security: compromised old shares cannot forge signatures after resharing.*

Theorem 4 (Ringtail Security). *Ringtail achieves existential unforgeability under the Module-LWE hardness assumption with parameters providing quantum security levels 1, 3, or 5 per NIST standards.*

7.3 Implementation Security

Constant-Time Arithmetic All cryptographic operations use constant-time implementations (saferith library) to prevent timing attacks.

Side-Channel Resistance

- No secret-dependent branches
- No secret-dependent memory access patterns

- Blinding of intermediate values

Zero-Knowledge Proofs All protocols use ZK proofs at critical steps:

- Π_{mod} : Paillier modulus is product of two primes
- Π_{prm} : Correct Paillier parameter generation
- Π_{sch} : Schnorr proof of knowledge
- Π_{affg} : Affine operation with group commitment
- Π_{log} : Discrete logarithm equality

7.4 Security Audit Findings

Third-party security audit (2024) identified and resolved:

1. **Nonce Generation:** Enhanced to hedged deterministic nonces
2. **Range Proofs:** Tightened bounds in ZK proofs
3. **Session Binding:** Strengthened session ID to prevent replay attacks
4. **Abort Handling:** Improved abort identification in CMP

All high and medium severity issues resolved.
Low severity recommendations implemented.

8 Deployment and Applications

8.1 Production Deployment

The framework is production-deployed securing:

- **Enterprise Custody:** Multi-billion dollar institutional custody with geographic distribution of signing parties
- **DeFi Protocols:** Decentralized autonomous organizations (DAOs) with on-chain governance
- **Cross-Chain Bridges:** Threshold-controlled bridge contracts securing cross-chain asset transfers
- **Wallet Abstraction:** Consumer-facing MPC wallets with mobile/web interfaces

8.2 Use Cases

Multi-Jurisdictional Custody 5-of-7 threshold with parties in different jurisdictions:

- Compliance: No single jurisdiction controls keys
- Disaster recovery: 2 party failures tolerable
- Geographic distribution: US, EU, Asia locations

DAO Treasury Management 7-of-11 threshold for large protocol treasuries:

- Security council members hold shares
- On-chain governance triggers signing
- LSS resharing for council elections

Institutional Trading 3-of-5 threshold for high-frequency trading:

- Compliance team: 2 shares
- Trading desk: 2 shares
- Risk management: 1 share
- CMP presignatures for low latency

Personal Custody 2-of-3 threshold for individual users:

- Mobile device: 1 share
- Hardware wallet: 1 share
- Cloud backup (encrypted): 1 share
- Doerner protocol for mobile-hardware signing

8.3 Integration Examples

XRPL Bridge

```
// 5-of-9 threshold controlling bridge account
config := cmp.Keygen(curve.Secp256k1{},
    selfID, parties, 4, pool)
```

```
// XRPL multi-sign transaction
xrpl := adapters.NewXRPLAdapter(
    adapters.SignatureECDSA, true)
digest, _ := xrpl.Digest(txBlob)

// Threshold signing with 5 parties
signature := cmp.Sign(config, signers,
    digest, pool)
encoded, _ := xrpl.Encode(signature)
```

Ethereum DAO Treasury

```
// 7-of-11 FROST threshold for gas efficiency
config := frost.Keygen(curve.Secp256k1{},
    selfID, parties, 6, pool)
```

```
// EIP-1559 transaction
eth := adapters.NewEthereumAdapter()
tx := &adapters.EIP1559Transaction{
    ChainID: big.NewInt(1),
```

```

    Nonce: 42,
    // ... other fields
}
digest, _ := eth.Digest(tx)

```

```

// 2-round FROST signing
signature := frost.Sign(config, signers,
    digest, pool)
encoded, _ := eth.Encode(signature)

```

Dynamic Validator Set

```

// Initial 5-of-9 validator set
oldConfigs := lss.Keygen(curve.Secp256k1{},
    selfID, oldParties, 4, pool)

// Add 3 new validators → 7-of-12
newParties := append(oldParties,
    newValidators...)
newConfigs := lss.Reshare(oldConfigs,
    newParties, 6, pool)

// Signing continues with new set
signature := lss.Sign(newConfigs[selfID],
    signers, message, pool)

```

8.4 Operational Metrics

Uptime 99.95% availability over 12-month period with:

- Automatic failover to backup parties
- LSS rollback for Byzantine fault recovery
- Zero downtime during membership changes

Signing Volume

- 2M+ signatures generated in production
- 500-2000 signatures/day across all deployments
- Peak load: 50 concurrent signing sessions

Error Rates

- Protocol failures: < 0.01% (mostly network timeouts)

- Identifiable aborts: 3 incidents (Byzantine parties excluded)
- LSS rollbacks: 12 automated recoveries

9 Comparison with Alternatives

Advantages

1. **Universal Chain Support:** Native adapters for 20+ blockchains vs. generic signature output requiring manual integration
2. **Dynamic Membership:** LSS live resharing vs. static key distribution
3. **Post-Quantum Ready:** Ringtail lattice signatures vs. elliptic-curve only
4. **Protocol Diversity:** Four complementary protocols vs. single implementation
5. **Production Validation:** 100% test coverage, security audit, real-world deployment

Trade-offs

1. **Complexity:** More features require larger codebase (vs. minimal research implementations)
2. **Performance:** ECDSA protocols slower than BLS (15ms vs. 5ms), but BLS lacks chain support
3. **Trust Assumptions:** LSS coordinator trusted for liveness (not secrecy)

10 Future Work

10.1 Short-Term Enhancements

Additional Protocols

- CGGMP21 updates (enhanced abort identification)
- SpeedyMuSig2 for fast 2-round Schnorr
- GG18-based EdDSA threshold signatures

System	Protocols	Chains	Dynamic	Post-Quantum	Signing	Pro
tss-lib [9]	GG18, GG20	Generic	No	No	60ms	1
ZenGo X [10]	GG20, EdDSA	5+	No	No	40ms	
Multi-Party-ECDSA [11]	Lindell 2P	Generic	No	No	30ms	R
Threshold-BLS [12]	BLS	Ethereum 2.0	No	No	5ms	
Lux Threshold	CMP, FROST, LSS, Doerner	20+	Yes (LSS)	Yes (Ringtail)	8-40ms	

Table 4: Comparison with existing threshold signature systems. Our framework uniquely combines multiple protocols, extensive chain support, dynamic membership, and post-quantum security in a production-ready package.

Chain Support Expansion

- Tier 2 chains: Cosmos, Polkadot, Avalanche, Aptos, Sui
- Privacy chains: Zcash, Monero (threshold view keys)
- Emerging chains: Movement, Monad, Berachain

Performance Optimization

- GPU acceleration for ZK proof generation
- WebAssembly compilation for browser deployment
- Presignature pool management with predictive allocation

10.2 Medium-Term Research

Asynchronous Protocols Remove need for synchronization:

- Asynchronous DKG (distributed key generation)
- Threshold signing with guaranteed termination
- Network partition tolerance

Proactive Security Enhancements

- Automated share refresh scheduling
- Mobile adversary resistance (adaptive corruption)
- Key escrow and recovery mechanisms

Advanced LSS Features

- Concurrent resharing without coordinator
- Threshold changes without full resharing
- Hierarchical threshold structures

10.3 Long-Term Vision

Fully Homomorphic Threshold Signatures

Enable threshold signing over encrypted data without revealing intermediate values.

Quantum-Safe Interoperability Hybrid classical/post-quantum schemes for gradual transition:

- Dual signatures (ECDSA + Ringtail)
- Merkle aggregation for efficiency
- Backward compatibility with existing chains

Decentralized Threshold Networks Public infrastructure for threshold signing-as-a-service:

- Economic incentives for liveness
- Slashing for Byzantine behavior
- Automated party discovery and reputation

AI Agent Integration Threshold signatures for AI economies:

- AI agents provisioned with single shares
- Multi-agent consent workflows
- “Know Your Agent” cryptographic identity

11 Conclusion

We presented a comprehensive universal threshold signature framework addressing practical deployment challenges across 20+ blockchains. Our system integrates four core protocols (CMP, FROST, LSS, Doerner) with chain-specific adapters providing native support for diverse signature schemes (ECDSA, EdDSA, Schnorr, post-quantum Ringtail).

The LSS dynamic resharing protocol enables live membership changes without downtime or master key reconstruction, solving critical operational challenges for production systems. Performance benchmarks demonstrate sub-25ms signing latency with 100% test coverage and Byzantine fault tolerance.

Production deployment secures billions in digital assets across enterprise custody, DeFi protocols, and cross-chain bridges, validating the framework’s practical viability. Post-quantum Ringtail signatures provide quantum-resistant alternatives for long-term security.

Over four years of continuous development (February 2021 - August 2025, 800+ commits), we evolved from initial CMP/FROST implementation to a production-ready universal infrastructure. The framework’s extensible architecture and comprehensive testing establish a foundation for next-generation threshold cryptography.

Acknowledgments

We thank the Lux Foundation for supporting this research. Implementation contributions from J.-P. Aumasson, A. Hamelink, and L. Meier on initial CMP/FROST protocols. Security audit by Trail of Bits. Deployment feedback from enterprise custody partners. LSS protocol co-development with Cornell University.

References

- [1] R. Canetti, R. Gennaro, S. Goldfeder, N. Makriyannis, and U. Peled. *UC Non-Interactive, Proactive, Threshold ECDSA*

with Identifiable Aborts. ACM CCS 2021. eprint.iacr.org/2021/060.

- [2] C. Komlo and I. Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. SAC 2020. eprint.iacr.org/2020/852.
- [3] V. J. Seesahai. *LSS MPC ECDSA: A Pragmatic Framework for Dynamic and Resilient Threshold Signatures*. Cornell University, August 2025.
- [4] J. Doerner, Y. Kondi, E. Lee, and a. shelat. *Secure Two-party Threshold ECDSA from ECDSA Assumptions*. IEEE S&P 2018. eprint.iacr.org/2018/499.
- [5] R. Gennaro and S. Goldfeder. *Fast Multi-party Threshold ECDSA with Fast Trustless Setup*. ACM CCS 2018.
- [6] R. Gennaro and S. Goldfeder. *One Round Threshold ECDSA with Identifiable Abort*. Cryptology ePrint Archive, 2020.
- [7] P. W. Shor. *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*. SIAM Journal on Computing, 1997.
- [8] P. Wuille, J. Nick, and A. Towns. *BIP-340: Schnorr Signatures for secp256k1*. Bitcoin Improvement Proposals, 2020.
- [9] Binance. *tss-lib: Threshold Signature Scheme library in Golang*. github.com/bnb-chain/tss-lib, 2019.
- [10] ZenGo X. *Multi-Party ECDSA/EdDSA Threshold Signature Scheme*. github.com/ZenGo-X/multi-party-ecdsa, 2020.
- [11] Y. Lindell. *Fast Secure Two-Party ECDSA Signing*. Journal of Cryptology, 2018.
- [12] Ethereum Foundation. *BLS Threshold Signatures for Ethereum 2.0*. Ethereum Research, 2020.

- [13] NIST. *Post-Quantum Cryptography Standardization*. National Institute of Standards and Technology, 2024.
- [14] L. Ducas et al. *Crystals-Dilithium: A Lattice-Based Digital Signature Scheme*. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2018.
- [15] Cronokirby. *saferith: Arithmetic with Better Side-Channel Resistance*. github.com/cronokirby/saferith, 2021.